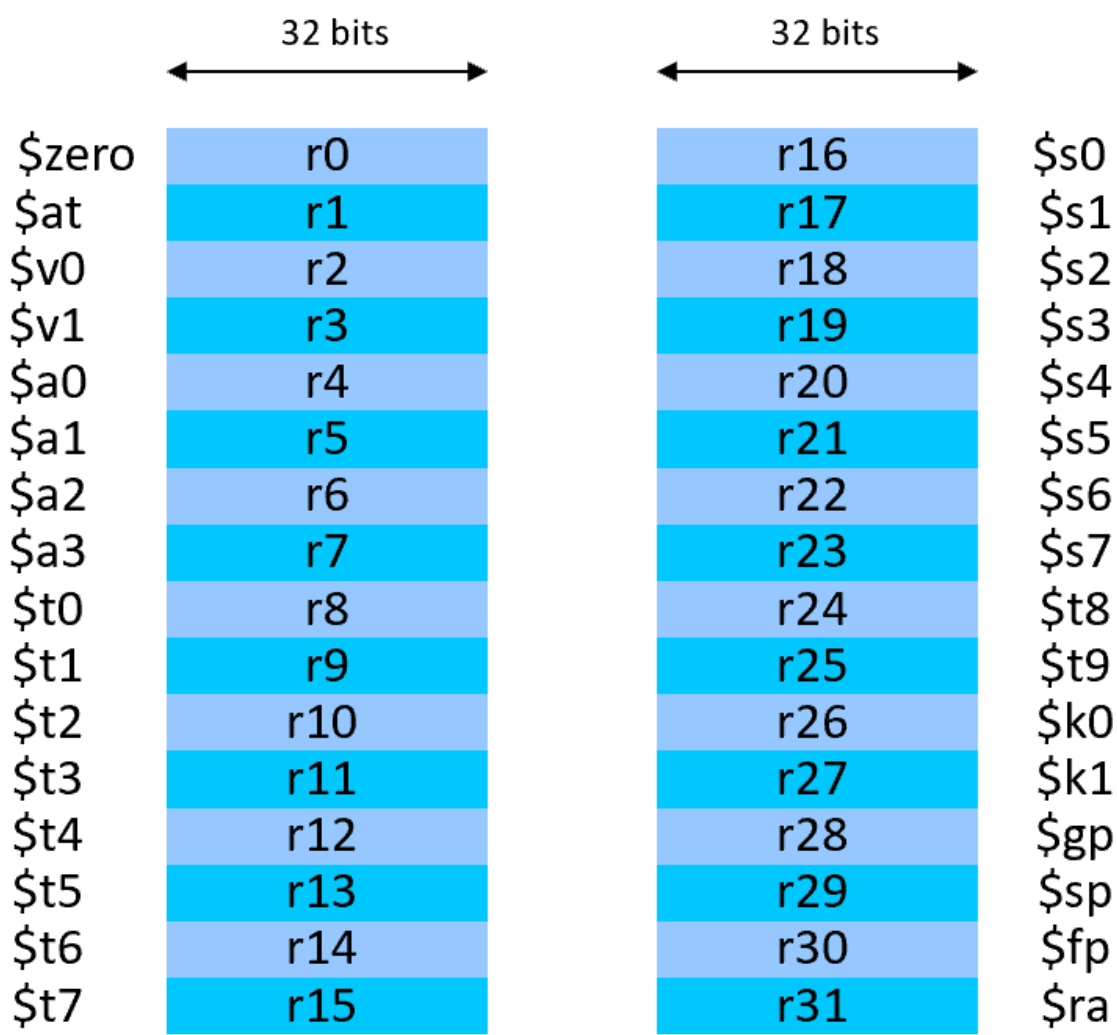


2.1.MIPS指令&MIPS汇编

指令格式：32位二进制数
指令简单，执行速度快
同样支持立即数

2.1.1寄存器

- MIPS有32个通用寄存器（编号 0-31），所有的加减法运算必须在寄存器里完成。
- 分类：
 - \$s0 - \$s7：变量寄存器（存C语言中的变量）。
 - \$t0 - \$t9：临时寄存器（存中间计算结果）。
 - \$zero：恒等于0（像数学里的0，非常实用）。
 - \$a0 - \$a3 和 \$v0 - \$v1：像快递盒，分别用来传参数和收返回值



General-Purpose Registers

寄存器	全称	寄存器号	主要作用
\$fp	Frame Pointer	\$s8 /30	栈帧指针，定位函数的局部变量和参数

寄存器	全称	寄存器号	主要作用
\$gp	Global Pointer	28	全局指针，定位全局变量和静态数据
\$sp	Stack Pointer	29	栈指针，用于函数调用和局部变量存储
\$at	Assembler Temporary	1	汇编器临时寄存器，用于伪指令的翻译
即在MIPS中进行静态数据访问时需要使用的寄存器是全局指针GP			
而SP：指向栈顶，栈变它就变			
FP：指向栈帧基准点，方便找局部变量和参数			

2.1.2需要补充的指令

类MOV指令

- lw: (Load Word): 内存->寄存器。 lw a,b 是b->a
 - sw: (Store Word): 寄存器->内存，格式： sw 寄存器,内存
- MIPS是按字节寻址的，存一个“字(word)”地址要加4
- CPU寄存器在前，RAM在后

决策指令：分支和跳转p.22

- beq/bne: 如果相等/不等就跳走，类似 JE 和 JNE
 - beq \$rs, \$rt, label, rs&rt是source寄存器，if相等就跳转到label对应
 - bne 同理
- slt :set on less than,如果第一个源操作数小于第二个，则目标寄存器设为1，反之为0.注意SLT是R-type, 也就是 slt rd, rs, rt 是比较rs和rt,设置rd. 和JB指令不同，slt不能跳转，需要搭配bne使用
- jr: jump with register content，根据寄存器的值跳转，可以起到类似 switch 的作用

立即数指令

- addi: 立即数加法， addi \$t, \$s, imm 即t =s + imm (立即数)
- li<寄存器>, <立即数>: 立即数赋值

2.1.3.指令组成部分与格式

Field size	6bits	5bits	5bits	5bits	5bits	6bits	All MIPS instruction 32 bits
R-format	op	<u>rs</u>	rt	rd	<u>shamt</u>	funct	Arithmetic instruction format
I-format	op	<u>rs</u>	<u>rt</u>	Imm/Word address			Data transfer ,branch format
J-format	op	target address (word)					Unconditional jump

Must bear in mind !

Region: ±2¹⁵

格式：

- R-格式：用于算术和逻辑运算指令。 add , and, slt
- I-格式：用于立即数操作、数据传输及条件分支指令。 addi , lw/sw , beq/bne

- **J-格式**：用于无条件跳转指令。

字段：【MIPS×机器语言】

1. **op (6 bits)：操作码字段**

- 定义了指令的基本操作类型。
- 类似于指令的“标识符”。
- 例子：add 的操作码是 0，lw 的操作码是 35，j 指令的操作码是 2。

2. **rs (5 bits)：源寄存器 1**

3. **rt (5 bits)：源寄存器 2**

4. **rd (5 bits)：目标寄存器**

- R-格式独有字段。
- 存储计算结果的目标寄存器。例如，通过加法将结果存入这个寄存器。

5. **shamt (5 bits)：移位量**

- R-格式独有字段。
- 用于移位指令，指定位移的位数（shift amount）。

6. **funct (6 bits)：功能字段**

- R-格式独有字段。
- 指定如加、减、或等特定操作，结合 op 决定最终的指令功能。

7. **Imm/Word address (16 bits)：立即数或偏移量字段**

- I-格式独有字段。
- 用于表示立即数或数据/分支指令中的偏移量。
- 范围： $\pm 2^{15}$ （2 字节），标注为 **Region: $\pm 2^{15}$** ，表示偏移地址通常以字节为单位，可访问正负 2^{15} 的地址范围。

8. **target address (26 bits)：目标地址字段**

- J-格式独有字段。
- 指定无条件跳转时的目标地址

示例：

```
Error: spawn pdf2svg ENOENT
```

2.1.4.C++转换MIPS汇编参考示例

示例：计算数组前N个元素的累加和

C++代码：

```
int sumArray(int arr[], int n) {  
    int sum = 0;  
    for (int i = 0; i < n; i++) {  
        sum = sum + arr[i];  
    }  
    return sum;  
}
```

对应的MIPS汇编：

```
# 函数原型: int sumArray(int arr[], int n)
# 参数: $a0 = arr数组首地址, $a1 = n
# 返回值: $v0

sumArray:
    # 初始化 sum = 0
    move $t0, $zero          # $t0 = sum = 0 (用$t0存sum)
    move $t1, $zero          # $t1 = i = 0 (用$t1存循环变量i)

# for循环判断: i < n
loop_check:
    slt $t2, $t1, $a1        # 如果 i 小于 n, 则将 $t2 设置为 1, 否则设置为 0
    beq $t2, $zero, loop_end  # 如果 i >= n, 跳出循环

    # 循环体: sum = sum + arr[i]
    # 计算 arr[i] 的地址: arr + i*4 (MIPS按字节寻址, int占4字节)
    sll $t2, $t1, 2          # $t2 = i * 4 (逻辑左移2位相当于乘4)
    add $t3, $a0, $t2         # $t3 = arr + i*4 (数组元素的地址)
    lw $t4, 0($t3)           # $t4 = arr[i] (从内存加载到寄存器)

    add $t0, $t0, $t4         # sum = sum + arr[i]
    addi $t1, $t1, 1          # i++

    j loop_check             # 跳转回循环判断

loop_end:
    # 返回 sum
    move $v0, $t0            # $v0 = sum
    jr $ra                   # 返回调用者
```

关键知识点解析：

MIPS指令	作用	解释
move \$t0, \$zero	寄存器赋值	将zero(0)移到t0，相当于 sum=0
sll \$t2, \$t1, 2	乘4运算	逻辑左移2位，等价于 i*4 ，用于计算数组偏移
addi \$t1, \$t1, 1	加立即数	循环变量i++
lw \$t4, 0(\$t3)	加载内存	从地址t3处加载一个int到t4
slt \$t2, \$t1, \$a1	比较大小	设置t2为1当i<\$n，否则为0
beq \$t2, \$zero, label	条件跳转	当条件不满足时跳出循环

寄存器分配总结：

- \$a0, \$a1：输入参数（数组首地址、数组长度n）
- \$t0：存储累加和sum
- \$t1：循环变量i
- \$t2, \$t3, \$t4：临时计算用
- \$v0：返回值
- \$at：用于展开伪指令，编译器在翻译伪指令时，需要一块"草稿纸"来暂存中间结果，\$at 就是这张草稿纸。它作为幕后临时工，用户程序不应当直接使用，否则会被编译器覆盖掉^57b80e

注意: 转MIPS代码涉及到修改S寄存器的情况就需要栈sp进行保护

因为s寄存器隐含"执行前后不能变"的条件.

栈的用法:

```
addi $sp, $sp, -4 ;开一个长度为4的栈,sp就是栈顶
sw    $s0, 0($sp) ;寄存器s0->栈sp+0
```

```
lw    $s0, 0($sp) ;反向操作
addi $sp, $sp, 4
```

为什么是sp-=4:首先32位MIPS一个变量是4个字节，所以一次性步长就是4，两个变量就是8

```
EntryNotFound (FileSystemError): Error: ENOENT: no such file or directory, open 'c:\Users\A\Documents\记事本.txt'
```

难点：递归函数转MIPS：

需要注意递归调用处需要用 jal 指令，因为要把返回地址保存到 \$ra ，如果直接用j回不来
因此栈需要存递归函数的参数**以及ra**两大部分

2.2.逻辑操作符

```
Error: spawn pdf2svg ENOENT
```

为什么是 NOR 而非 NOT ?因为要满足MIPS三操作数的要求，而 $\text{NOR}(x, x) \equiv \neg(x \vee x) \equiv \neg x$,在两个X外增添一个destination操作数就是三操作数

2.3.MIPS&函数/过程调用

p.26

2.3.1.相关寄存器

- 参数寄存器： a0~a3
- 返回值寄存器： v0~v1
- 返回地址寄存器： ra

2.3.2.相关指令

```
jal Address #类似汇编的CALL，跳转到子程序/函数的地址，把当前地址放到$ra中
jr $ra      #类似汇编的RET，返回断点
```

2.3.3.使用更多寄存器/栈相关

p.27

涉及到堆栈指针 \$sp

2.3.4.嵌套/递归调用

p.29

2.4.人机交互

2.4.1.相关MIPS指令

p.34

- lb : load byte
- sb : Store Byte, 将寄存器的低8位存回内存的指定字节位置。
- lbu : Load Byte Unsigned, 从内存加载一个无符号字节到寄存器并将高位填0, 零扩展为32位。
- lhu : Load Halfword Unsigned, 从内存加载无符号半字 (16位) 到寄存器并将高位填0, 零扩展为32位。
- lui : Load Upper Immediate, 将一个16位立即数加载到寄存器的高16位, 低16位填充为0, 常与 ori 配合生成32位常量。

2.4.2.寻址方式

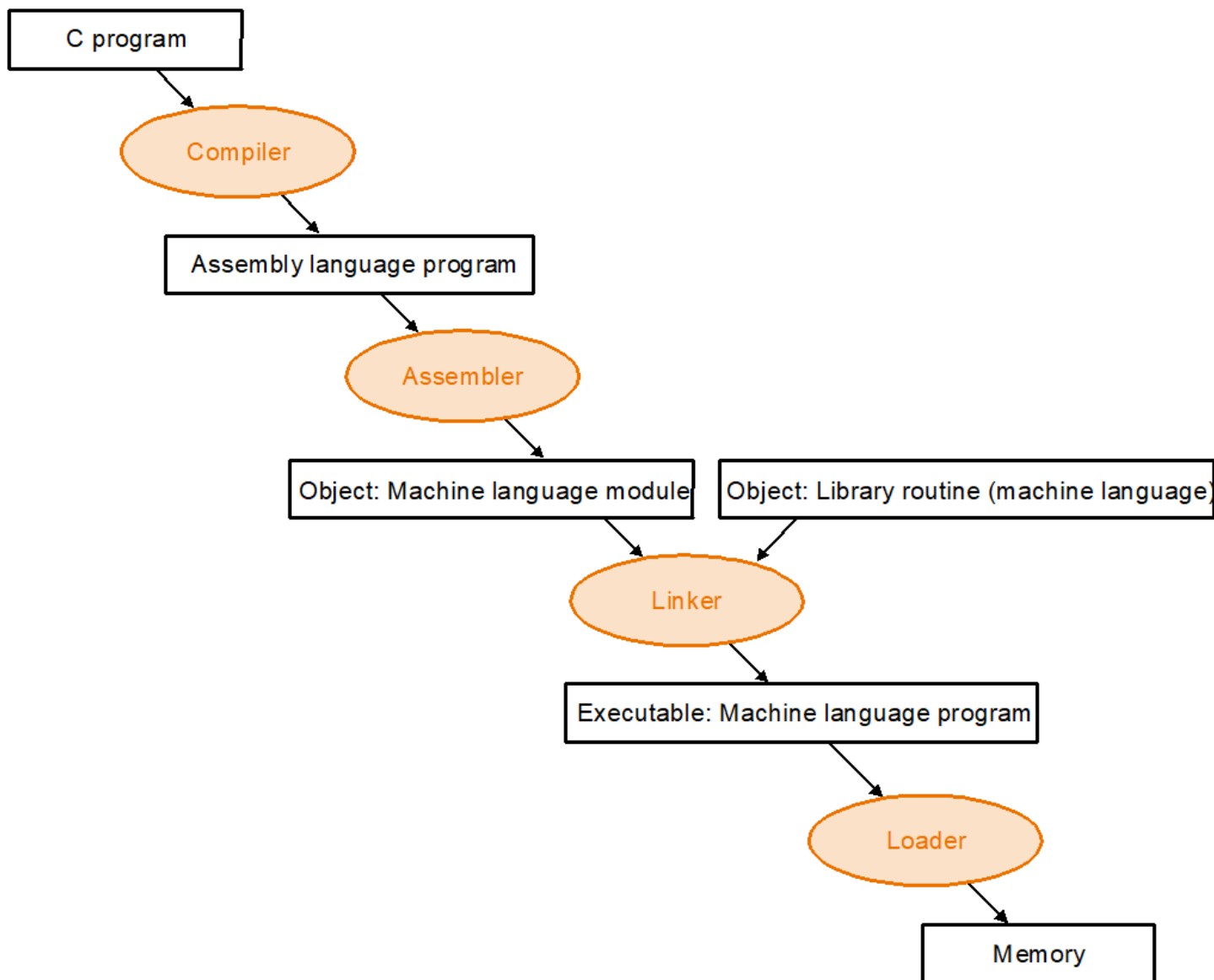
p.37

Error: spawn pdf2svg ENOENT

- 立即数寻址: addi
- 寄存器寻址: add
- 基址寻址: 操作数在内存中, lw
- PC相对寻址: beq 新的 PC 值 = (PC + 4) + (分支偏移地址左移2位)
 - 分支偏移地址是 (L1目标地址-(PC+4))/4
- 伪直接寻址: j Address 26位的字地址左移两位后直接拼接 (PC+4) 中的高4位最终形成32位跳转地址。
**条件分支的地址范围: 分支前后地址范围各大约128K—— 2^{18}

2.4.3.机器语言解码

编译-汇编-链接-加载



2.4.4.大端vs小端



以0xabcdef12为例：

地址	0	1	2	3
大端模式	ab	cd	ef	12
小端模式	12	ef	cd	ab

这么记：大端是高位数据存低地址，正向呈现；小端是高位数据存高地址，反向呈现

2.5.32位立即数的装入

例子：将一个32位常数0x002B8012加载到寄存器\$S0 中

方案1：lui+ori

```
lui $s0, 0x002B    lui $s0, 0x002B ori $s0, $s0, 0x8012
```

lui \$reg,imm_high_half 指令把32位立即数imm的高16位装入32位寄存器reg的高半部分，并把低半部分清0

ori \$reg,\$reg,imm_low_half 指令把立即数低16位与0做按位or，以装入

方案2：li

li 是伪指令，汇编器会将 li \$s0, 0x002B8012 展开为：

```
lui $s0, 0x002B
ori $s0, $s0, 0x8012
```

不可行：lui+addi

```
lui $s0, 0x002B
addiu $s0, $s0, 0x8012
```

addi会对0X8012做符号扩展，高16位全部是1

X.错题

1.

12. 以下描述不正确的是（ ）。↵

- A. 如果函数不使用大量局部数据时不需创建活动帧↵
- B. 活动帧由函数通过调整 $\$sp$ 来创建↵
- C. 可以只通过 $\$sp$ 来使用活动帧，而不需要使用 $\$fp$ ↵
- D. 函数结束前需释放分配的活动帧↵

答案选A。

首先活动帧就是栈的一部分【可以直接理解为栈】。A选项当有递归调用时，至少 $\$ra$ 需要开栈保存，此时虽然没有大量局部数据，仍需创建活动帧；B是对的，开栈的一种方式就是 $sp-=x$ ，C也是对的，我们C++转MIPS的题要开栈，单用不到fp。D就显而易见了。